SPANS-04: Service Dependencies & Flow Analysis

> **Series:** SPANS — Distributed Tracing and Spans | **Notebook:** 4 of 8 | **Created:** December 2025 | **Last Updated:** 10/02/2026

Mapping Your Distributed System

This notebook teaches you how to use span data to understand service relationships, analyze request flows, and identify critical dependencies in your system.

Table of Contents

1. Understanding Service Topology
2. Discovering Services
3. Mapping Service Dependencies
4. Client-Server Call Patterns
5. Async Messaging Flows
6. Trace Hierarchy Analysis
7. Cross-Service Latency Analysis
8. Critical Path Analysis

Prerequisites

Before starting this notebook, ensure you have:

- ✅ Completed **SPANS-01** through **SPANS-03**

- ✅ Access to a Dynatrace environment with distributed trace data
- ✅ Understanding of span kinds (server, client, producer, consumer)

SaaS 1.337: OpenTelemetry `service.name` in SDv1 Service Names

Per the SaaS 1.337 release notes, *"You can now use the OTel service.name attribute to customize the name of SDv1 services."* Smartscape shows the result as `service.name (detected name)`, and the change *"affects the service name in all Smartscape use cases, and the dt.service.name attribute on spans and metrics."*

So `dt.service.name` — present on every span, OneAgent and OpenTelemetry alike — carries the enriched name where an OTel `service.name` exists, and the detected name otherwise. That makes it the service field this series groups by:

```
fetch spans, from:-1h
| filter isNotNull(dt.service.name)
| summarize span_count = count(), by:{dt.service.name, dt.smartscape.service}
| sort span_count desc
| limit 50
```

> **Sources:** SaaS 1.337 release notes (DT docs). **Dictionary:** `dt.service.name ( stable ;`
>
> described as equal to the Smartscape service node name), read 10/02/2026.

SaaS 1.344: Frontend nodes in the service dependency graph

SaaS 1.344+ (staged tenant rollout from 07/29/2026). Per the release notes, *"the Service dependency graph now shows FRONTEND nodes"*, and spans with frontend links can be followed into the related user events and sessions from Distributed Tracing.

Two topology-facing changes:

1. **The Smartscape service dependency graph now displays `FRONTEND` nodes.** The frontend that originates a request appears as a node in its own right rather than as an implicit off-graph caller — the topology view starts where the user does, instead of at the first server span it can see.
2. **Distributed tracing can explore frontend events and sessions without switching apps.** A slow backend dependency and the user session that hit it can be read on one surface.

Neither change alters any DQL in this notebook. The `span.kind == "client"` / `server.address` techniques in **Section 3** remain how you *derive* dependencies from span data — and on a pre-1.344 tenant they remain the only way to do it. The graph is a reading surface layered over the same relationships; it does not replace the queries, and a dependency you need in a dashboard tile, a report, or an SLO still has to be queried.

The frontend-to-backend link in span data

The linkage the graph draws on is modelled in the semantic dictionary as **`frontend_link`** and exposed on spans through the **`frontend.link`** record field:

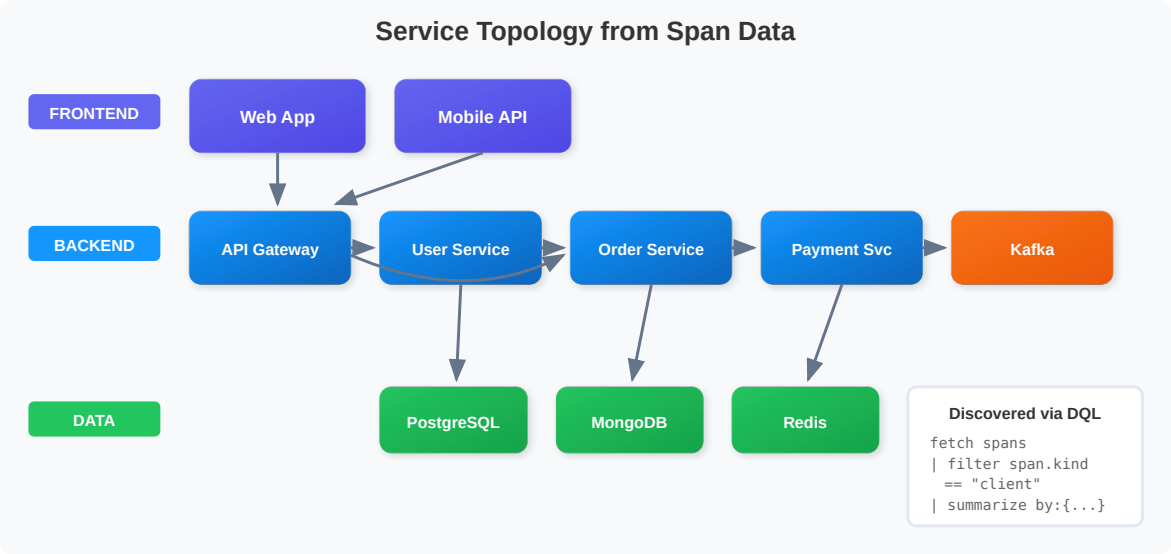
Field	Carries
`span.id`	The backend span the frontend action links to
`dt.rum.session.id`	The RUM session that originated the request
`dt.rum.instance.id`	The RUM instance (page / view load) within that session
`dt.rum.is_linking_candidate`	Whether the span is eligible for frontend linking

Session-side configuration of this linkage belongs to the **WEBRUM** series. For how the session identifier travels between browser and backend, see **OTEL-04 § 4 — Context Propagation**.

---

1. Understanding Service Topology

Service topology shows how your services connect and communicate:



Key Span Types for Topology

Span Kind	Role	What It Tells You
`server`	Receives requests	Entry points, inbound traffic
`client`	Makes requests	Outbound calls, dependencies
`producer`	Sends messages	Async event sources
`consumer`	Receives messages	Async event handlers

---

2. Discovering Services

First, discover all services generating spans in your environment:

```
// List all services with span counts
fetch spans, from:-1h
| summarize {
    span_count = count(),
    operations = countDistinct(span.name)
  }, by: {dt.service.name}
| sort span_count desc
| limit 30
```

```
// Service role analysis - understand each service's function
fetch spans, from:-1h
| summarize {
    server_spans = countIf(span.kind == "server"),
    client_spans = countIf(span.kind == "client"),
    internal_spans = countIf(span.kind == "internal"),
    producer_spans = countIf(span.kind == "producer"),
    consumer_spans = countIf(span.kind == "consumer")
  }, by: {dt.service.name}
| sort server_spans desc
| limit 30
```

```
// Discover all operations/endpoints per service
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {
    request_count = count(),
    avg_duration_ms = avg(duration) / 1ms
  }, by: {dt.service.name, span.name}
| sort dt.service.name asc, request_count desc
| limit 50
```

---

3. Mapping Service Dependencies

Use CLIENT spans to understand which services call which other services:

Key Attributes for Dependencies

Attribute	Description
`server.address`	Target host/address (`stable`)
`server.port`	Target port (`stable`)

`peer.service`, which older material uses for the target service, has no row in the semantic dictionary and was set on 0 of 8,286,060 spans over 24 hours on the validation tenant. Map dependencies from `server.address` on client spans, or by joining each client span to the server span it called (same `trace.id`, server `span.parent_id` = client `span.id`). The join below groups by service **name** for readability; service names can

repeat across applications (`frontend` was 4 services on the validation tenant), so add `dt.smartscape.service` on both sides when one name covers several services.

> **Dictionary:** `server.address` (`stable`), `server.port` (`stable`); no row for `peer.service` or `http.host` , read 10/02/2026 (control: `server.address` returned a row).

```
// Map service-to-service calls using CLIENT spans
fetch spans, from:-1h
| filter span.kind == "client"
| summarize {
    call_count = count(),
    avg_latency_ms = avg(duration) / 1ms,
    error_count = countIf(span.status_code == "error")
  }, by: {dt.service.name, span.name}
| fieldsAdd error_rate_pct = (error_count * 100.0) / call_count
| sort call_count desc
| limit 50
```

```
// Which service calls which: join each client span to the server span it
called.
// The server span's parent is the client span (same trace.id).
fetch spans, from:-1h
| filter span.kind == "server" and isNotNull(span.parent_id)
| fields trace.id, callee = dt.service.name, parent = span.parent_id
| join [
    fetch spans, from:-1h
    | filter span.kind == "client"
    | fields trace.id, caller = dt.service.name, client_span = span.id,
call_duration = duration
  ], on:{trace.id, left[parent] == right[client_span]}, fields:{caller,
call_duration}
| filter caller != callee
| summarize {call_count = count(), avg_latency_ms = avg(call_duration) / 1ms},
by:{caller, callee}
| sort call_count desc
| limit 30
```

```
// Map dependencies using HTTP host/URL information
fetch spans, from:-1h
| filter span.kind == "client" and isNotNull(server.address)
| summarize {
    call_count = count(),
    avg_latency_ms = avg(duration) / 1ms,
    error_count = countIf(span.status_code == "error")
  }, by: {dt.service.name, server.address}
| fieldsAdd error_rate_pct = (error_count * 100.0) / call_count
| sort call_count desc
| limit 30
```

---

4. Client-Server Call Patterns

Analyze the matching CLIENT and SERVER span pairs to understand inter-service communication:

```
// Analyze outbound calls from each service
fetch spans, from:-1h
| filter span.kind == "client"
| summarize {
    outbound_calls = count(),
    avg_latency_ms = avg(duration) / 1ms,
    p99_latency_ms = percentile(duration, 99) / 1ms,
    error_count = countIf(span.status_code == "error")
  }, by: {dt.service.name}
| fieldsAdd error_rate_pct = (error_count * 100.0) / outbound_calls
| sort outbound_calls desc
| limit 20
```

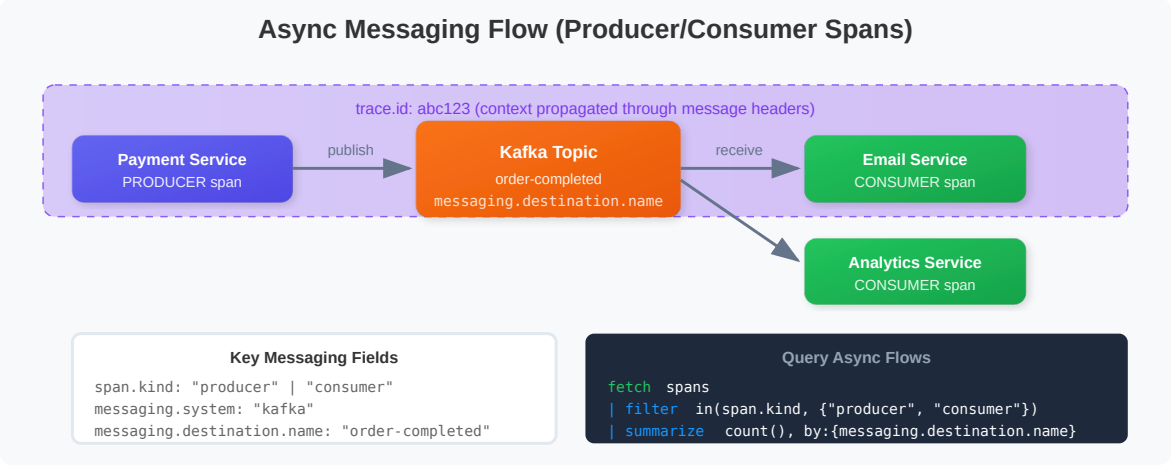
```
// Analyze inbound requests to each service
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {
    inbound_requests = count(),
    avg_latency_ms = avg(duration) / 1ms,
    p99_latency_ms = percentile(duration, 99) / 1ms,
    error_count = countIf(span.status_code == "error")
  }, by: {dt.service.name}
| fieldsAdd error_rate_pct = (error_count * 100.0) / inbound_requests
| sort inbound_requests desc
| limit 20
```

```
// Find services with high outbound/inbound ratio (integration heavy)
fetch spans, from:-1h
| summarize {
    inbound = countIf(span.kind == "server"),
    outbound = countIf(span.kind == "client")
  }, by: {dt.service.name}
| filter inbound &gt; 0
| fieldsAdd outbound_ratio = toDouble(outbound) / toDouble(inbound)
| sort outbound_ratio desc
| limit 20
```

---

5. Async Messaging Flows

Analyze asynchronous messaging patterns using PRODUCER and CONSUMER spans:



Key Messaging Attributes

Attribute	Description
`messaging.system`	Kafka, RabbitMQ, etc.
`messaging.destination.name`	Topic/queue name
`messaging.operation.type`	publish, receive, etc. (`messaging.operation` has no dictionary row)

Trace Context Across Message Brokers

PRODUCER and CONSUMER spans only join the **same trace** if trace context travels inside the message. Whether that happens automatically depends on the client library:

Broker	Auto-instrumentation	Implication
Kafka, RabbitMQ	First-class — OpenTelemetry Java lists Kafka `0.11+` and RabbitMQ `2.7+`	`traceparent` is injected and extracted for you
Google Cloud Pub/Sub	**Absent from that supported-libraries list**	Context is **not** propagated for you — the trace breaks silently at each hop

Where auto-instrumentation is missing, carry the W3C `traceparent` yourself through the message `attributes` map:

- **Publisher:** inject the active context as a `traceparent` attribute before publishing.
- **Subscriber:** read `traceparent` from the message attributes and use it as the parent when starting the consumer span.

Symptom to look for: consumer spans that are *root* spans. If the queries below show healthy producer **and** consumer volume for a destination but the two never appear in one trace, missing context propagation is the usual cause.

> Auto-instrumentation coverage varies by language and client version — verify for your own runtime rather than assuming. **Sources:** Span and trace context propagation (DT docs); OpenTelemetry Java supported libraries (OpenTelemetry GitHub)

```
// Find message producers (services sending async messages)
fetch spans, from:-1h
| filter span.kind == "producer"
| summarize {
    messages_sent = count(),
    avg_duration_ms = avg(duration) / 1ms,
    error_count = countIf(span.status_code == "error")
  }, by: {dt.service.name, span.name}
| sort messages_sent desc
| limit 20
```

```
// Find message consumers (services receiving async messages)
fetch spans, from:-1h
| filter span.kind == "consumer"
| summarize {
    messages_received = count(),
    avg_processing_ms = avg(duration) / 1ms,
    error_count = countIf(span.status_code == "error")
  }, by: {dt.service.name, span.name}
| fieldsAdd error_rate_pct = (error_count * 100.0) / messages_received
| sort messages_received desc
| limit 20
```

```
// Analyze messaging system usage (Kafka, RabbitMQ, etc.)
fetch spans, from:-1h
| filter span.kind == "producer" or span.kind == "consumer"
| filter isNotNull(messaging.system)
| summarize {
    message_count = count(),
    producers = countIf(span.kind == "producer"),
    consumers = countIf(span.kind == "consumer"),
    error_count = countIf(span.status_code == "error")
  }, by: {messaging.system, messaging.destination.name}
| sort message_count desc
| limit 20
```

```
// Map producer to consumer relationships
fetch spans, from:-1h
| filter span.kind == "producer" or span.kind == "consumer"
| summarize {
    span_count = count()
  }, by: {dt.service.name, span.kind, messaging.destination.name}
| sort messaging.destination.name, span.kind
| limit 30
```

6. Trace Hierarchy Analysis

Analyze parent-child relationships within traces to understand call depth:

```
// Count spans per trace to understand trace complexity
// trace_duration_ms is wall-clock (last end minus first start), not a sum of
span durations.
fetch spans, from:-1h
| summarize {
    span_count = count(),
    services_involved = countDistinct(dt.service.name),
    first_start = min(start_time),
    last_end = max(end_time)
  }, by: {trace.id}
| fieldsAdd trace_duration_ms = (last_end - first_start) / 1ms
| fields trace.id, span_count, services_involved, trace_duration_ms
| sort span_count desc
| limit 25
```

```
// Examine a complete trace hierarchy
// Replace YOUR_TRACE_ID with an actual trace ID from above
fetch spans, from:-1h
// trace.id is a uid: keep the toUid() wrapper — a plain string never matches.
// | filter trace.id == toUid("YOUR_TRACE_ID")
| fieldsAdd duration_ms = duration / 1ms
| fields start_time,
         span.id,
         span.parent_id,
         dt.service.name,
         span.name,
         span.kind,
         duration_ms
| sort start_time asc
| limit 100
```

```
// Find entry points (root spans) and their downstream services
fetch spans, from:-1h
| filter isNull(span.parent_id)
| summarize {
    entry_count = count(),
    avg_duration_ms = avg(duration) / 1ms
  }, by: {dt.service.name, span.name}
| sort entry_count desc
| limit 20
```

---

7. Cross-Service Latency Analysis

Identify latency hot spots between services:

```
// Latency by service-to-service call
fetch spans, from:-1h
| filter span.kind == "client" and isNotNull(server.address)
| summarize {
    call_count = count(),
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms,
    p99_ms = percentile(duration, 99) / 1ms
  }, by: {dt.service.name, server.address}
| sort p95_ms desc
| limit 20
```

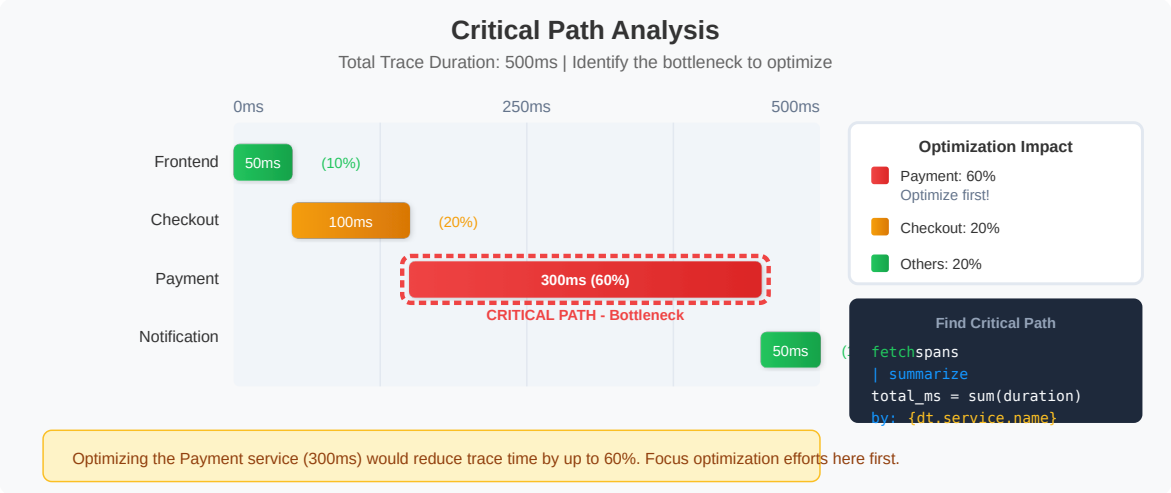
```
// Time spent per service in traces
fetch spans, from:-1h
| summarize {
    total_time_ms = sum(duration) / 1ms,
    span_count = count(),
    avg_per_span_ms = avg(duration) / 1ms
  }, by: {dt.service.name}
| sort total_time_ms desc
| limit 20
```

```
// Find slowest dependencies (CLIENT spans)
fetch spans, from:-1h
| filter span.kind == "client"
| filter duration &gt; 500ms
| summarize {
    slow_call_count = count(),
    avg_duration_ms = avg(duration) / 1ms,
    max_duration_ms = max(duration) / 1ms
  }, by: {dt.service.name, span.name}
| sort avg_duration_ms desc
| limit 20
```

---

8. Critical Path Analysis

Identify the services and operations that contribute most to end-to-end latency:



```
// Find services contributing most span time.
// sum(duration) is INCLUSIVE time: a span's duration includes the child calls
it waits on,
// so a caller is credited with its callees' time as well. Use it to rank
where time is
// spent, not as each service's own (self) time.
fetch spans, from:-1h
| summarize {
    total_span_time_ms = sum(duration) / 1ms,
    span_count = count(),
    avg_duration_ms = avg(duration) / 1ms,
    max_duration_ms = max(duration) / 1ms
  }, by: {dt.service.name}
| sort total_span_time_ms desc
| limit 15
```

```
// Find slowest operations across all services
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {
    call_count = count(),
    avg_duration_ms = avg(duration) / 1ms,
    p99_duration_ms = percentile(duration, 99) / 1ms,
    total_time_ms = sum(duration) / 1ms
  }, by: {dt.service.name, span.name}
| filter call_count &gt; 10
| sort p99_duration_ms desc
| limit 20
```

```
// Identify high-impact optimization candidates
// (high volume + high latency = most benefit from optimization)
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {
    call_count = count(),
    avg_duration_ms = avg(duration) / 1ms,
    total_time_ms = sum(duration) / 1ms
  }, by: {dt.service.name, span.name}
| filter call_count &gt; 50
| fieldsAdd impact_score = call_count * avg_duration_ms
| sort impact_score desc
| limit 20
```

Summary

In this notebook, you learned:

✅ **Service discovery** - Find all services and their operations
✅ **Dependency mapping** - Use CLIENT spans with `server.address`, or join client spans to the server spans they called
✅ **Client-server patterns** - Analyze inbound/outbound call ratios
✅ **Async messaging** - Track PRODUCER/CONSUMER spans through message queues
✅ **Trace hierarchy** - Understand span parent-child relationships
✅ **Cross-service latency** - Find latency hot spots between services
✅ **Critical path analysis** - Identify bottlenecks for optimization

---

Next Steps

Continue to **SPANS-05: Advanced Span Analytics** to learn:
- Time series analysis and trending
- Complex aggregations and calculations
- Building dashboard-ready queries
- Alerting patterns

References

- Trace semantic conventions (DT docs)
- SaaS 1.337 release notes (DT docs)
- SaaS 1.344 release notes (DT docs)
- Span and trace context propagation (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.